🚀 OBJECT ORIENTED SYSTEMS (OOS) — 2023 QUESTION 1 COMPLETE DETAILED SOLUTION

🌟 Introduction

Question 1 of the 2023 Object Oriented Systems examination belongs to CO1 and focuses on some of the most fundamental yet extremely important concepts of Java programming such as:

- 🚀 Classes and Objects
- 🧠 Method Overloading and Overriding
- 💡 Constructors
- 🔥 Inheritance
- ⚠️ Abstract Classes
- 📘 Arrays
- 🧿 Static and Non-static behavior
- 🎯 Interfaces
- 🔁 Multi-level Inheritance

These concepts form the foundation of Object-Oriented Programming in Java. Without mastering them properly, advanced topics such as multithreading, reflection, generic programming, design patterns, and enterprise Java development become extremely difficult.

This question is highly conceptual and tests both:

- Java syntax understanding
- Deep OOP behavior analysis.

📄 QUESTION 1(a)(i)

❓ **Analyze the following code and choose the correct option.**

```
class Test {
    int i;
}

class Main {
    public static void main(String args[]) {
        Test t;
        System.out.println(t.i);
    }
}
```

Options:

- A) 0
- B) Garbage value

- C) Compiler error
- D) Runtime error

✅ Answer

The correct answer is: ✅ **C) Compiler Error**

🌟 Detailed Explanation

In Java, local reference variables are NOT automatically initialized. Here:

```
Test t;
```

declares only a reference variable. No object is created. Thus:

```
t.i
```

tries to access instance variable `i` using an uninitialized reference. Since `t` does not point to any object, the compiler generates an error.

🌟 Important Concept

Java distinguishes between declaration and object creation. **Declaration:**

```
Test t;
```

only creates a reference variable. **Object creation requires:**

```
Test t = new Test();
```

🌟 Correct Program

```
class Test {
    int i;
}

class Main {
    public static void main(String args[]) {
        Test t = new Test();
        System.out.println(t.i);
    }
}
```

🖥️ **Output**

```
0
```

🌟 Why Output is 0?

Instance variables receive default values automatically. For `int i;`, the default value becomes `0`.

🎯 Conclusion

A reference variable must point to a valid object before accessing instance members. Otherwise, a compiler error occurs because Java prevents unsafe memory access 🚀.

📃 QUESTION 1(a)(ii)

❓ **What is the output of the following JAVA program?**

```
class Test {
    public static void main(String[] args) {
        Test obj = new Test();
        obj.start();
    }

    void start() {
        String stra = "do";
        String strb = method(stra);
        System.out.print(" " + stra + strb);
    }

    String method(String stra) {
        stra = stra + "good";
        System.out.print(stra);
        return "good";
    }
}
```

✅ Answer

The correct output is: dogood dogood

🌟 Detailed Explanation

This question mainly tests String immutability, parameter passing, local variable behavior, and method execution flow 🧠.

🌟 **Step-by-Step Execution** Initially:

```
String stra = "do";
```

Thus: `stra = "do"`

🔹 **Method Call**

```
String strb = method(stra);
```

passes `"do"` to the method. Inside the method:

```
stra = stra + "good";
```

creates a NEW string because Strings are immutable in Java. Thus local `stra` becomes `"dogood"`.

Then:

```
System.out.print(stra);
```

prints: `dogood`

Method returns:

```
return "good";
```

Thus: `strb = "good"`

🔹 **Back to start()** Original `stra` remains unchanged because Strings are immutable. Thus: `stra = "do"` and `strb = "good"`

Finally:

```
System.out.print(" " + stra + strb);
```

prints: `dogood`

Thus total output becomes: `dogood dogood`

🌟 Important Concept — String Immutability

Strings cannot be modified directly. Whenever modification occurs, a new `String` object is created. The original object remains unchanged. This provides security, thread safety, and immutability guarantees.

🎯 Conclusion

This program demonstrates how Java handles immutable String objects and method parameter passing 🚀.

🗒️ QUESTION 1(a)(iii)

❓ Choose the correct option(s)

```
class Base {
    final public void show() {
        System.out.println("Base::show() called");
    }
}

class Derived extends Base {
    public void show() {
        System.out.println("Derived::show() called");
    }
}
```

✅ Answer

✅ **Compiler Error**

🌟 Detailed Explanation

The keyword `final` prevents method overriding. Thus:

```
final public void show()
```

means child classes CANNOT redefine this method. However, the `Derived` class attempts:

```
public void show()
```

which is illegal. Therefore the compiler generates an error ⚠️.

🌟 Purpose of final Methods

Final methods provide security, implementation stability, and prevention of unwanted modification. Example: important framework methods, core library behaviors.

🌟 Correct Approach

Either remove `final`, or do not override the method.

🎯 Conclusion

A `final` method cannot be overridden because Java protects its implementation from modification 🚀.

📄 QUESTION 1(a)(iv)

❓ **Predict the output of the following Java program.**

```
public class T {
    int t = 20;

    T() {
        t = 40;
    }
}

class Main {
    public static void main(String args[]) {
        T t1 = new T();
        System.out.println(t1.t);
    }
}
```

✅ Answer

`40`

🌟 Detailed Explanation

Initially instance variable `int t = 20;` gets initialized. Then the constructor executes:

```
T() {
    t = 40;
}
```

The constructor overrides the previous value. Thus the final value becomes `40`.

🌟 Important Concept

Object creation occurs in phases:

1. Memory allocation
2. Default initialization
3. Instance variable initialization
4. Constructor execution

The constructor executes LAST. Thus the constructor value dominates.

🎯 Conclusion

Constructors can modify instance variables after initialization, allowing object-specific customization 🚀.

🗒️ QUESTION 1(a)(v)

❓ Which of the following is FALSE about abstract classes in Java?

Options:

- A) If derived class does not implement all abstract methods, it must also be abstract.
- B) Abstract classes can have constructors.
- C) A class can inherit from multiple abstract classes.
- D) None of the above.

✅ Answer

✅ C) A class can inherit from multiple abstract classes.

🌟 Detailed Explanation

Java does NOT support multiple inheritance of classes. Thus:

```
class A extends B, C
```

is illegal. Even if `B` and `C` are abstract, multiple inheritance still remains prohibited ⚠️.

🌟 Why Java Avoids Multiple Inheritance?

To prevent ambiguity, the diamond problem, and method conflicts.

🌟 Important Properties of Abstract Classes

Abstract classes:

- may contain constructors
- may contain concrete methods
- may contain abstract methods

✅ Example:

```
abstract class Shape {
    Shape() {
        System.out.println("Constructor");
    }

    abstract void draw();
}
```

🎯 Conclusion

Java allows inheritance from only ONE class whether abstract or concrete 🚀.

📄 QUESTION 1(a)(vi)

❓ Analyze the following code and choose the correct option.

```
class Test {
    public static void main(String args[]) {
        int arr[] = new int[2];
        System.out.println(arr[0]);
        System.out.println(arr[1]);
    }
}
```

✅ Answer

```
0
0
```

🌟 Detailed Explanation

Arrays in Java are objects. When an array is created (`new int[2]`), memory is allocated dynamically. All integer elements automatically receive the default value `0`.

Thus, `arr[0]` and `arr[1]` both print `0`.

🌟 Important Concept

Unlike C/C++, Java arrays NEVER contain garbage values. Java automatically initializes array elements. This improves security, reliability, and predictability.

🎯 Conclusion

Java arrays automatically initialize primitive values using datatype-specific defaults 🚀.

📄 QUESTION 1(b)

❓ Consider a case of multi-level inheritance where a class Base is inherited by Child which is again inherited by GrandChild. Suppose Base has a parameterized constructor and a method display().

✅ Answer

Multilevel inheritance means the inheritance chain continues across multiple generations. Example: `Base → Child → GrandChild`

Here:

- `Child` inherits `Base`
- `GrandChild` inherits `Child`

Thus `GrandChild` indirectly inherits `Base` also 🚀.

🌟 Java Program

```
class Base {
    int x;

    Base(int x) {
        this.x = x;
    }

    void display() {
        System.out.println("x = " + x);
    }
}

class Child extends Base {
    Child(int x) {
        super(x);
    }
}

class GrandChild extends Child {
    GrandChild(int x) {
        super(x);
    }
}

class Main {
    public static void main(String args[]) {
        GrandChild g = new GrandChild(100);
        g.display();
    }
}
```

🖥️ Output

```
x = 100
```

🌟 Deep Explanation

Since `Base` has a parameterized constructor `Base(int x)`, `Child` must invoke it using `super(x);`. Similarly, `GrandChild` calls the `Child` constructor. Thus constructor chaining occurs. Finally, `GrandChild` inherits:

- `Base` variables
- `Base` methods
- `Child` behaviors

🎯 Conclusion

Multilevel inheritance enables hierarchical code reuse and constructor chaining across multiple generations 🚀.

📄 QUESTION 1(c)

❓ What is the purpose of using interfaces in Java? ❓ Consider two interfaces IN1 and IN2 each having display() method. Create class Sample implementing both interfaces. ❓ Show how display() method of IN1 can be invoked using object of Sample class.

✅ Answer

Interfaces provide abstraction and support multiple inheritance in Java. An interface contains abstract methods, constants, and behavioral contracts. Interfaces specify WHAT to do, not HOW to do it.

🌟 Why Interfaces are Needed?

Interfaces help achieve:

- Abstraction
- Loose coupling
- Multiple inheritance
- Polymorphism

✅ Java Program

```
interface IN1 {
    void display();
}

interface IN2 {
    void display();
}

class Sample implements IN1, IN2 {
    public void display() {
        System.out.println("Display Method");
    }
}

class Main {
    public static void main(String args[]) {
        IN1 obj = new Sample();
        obj.display();
```

```
    }
}
```

🖥️ **Output**

```
Display Method
```

🌟 Deep Explanation

`Sample` implements both interfaces. Since both contain `display()`, only one implementation is required. Object creation:

```
IN1 obj = new Sample();
```

uses an interface reference. Thus runtime polymorphism, abstraction, and loose coupling are achieved 🚀.

🎯 Conclusion

Interfaces provide abstraction and multiple inheritance support, making Java systems more flexible and modular.

📄 QUESTION 1(d)

❓ **Discuss the significance of each word in:** `public static void main(String args[])`

✅ Answer

The statement `public static void main(String args[])` is the entry point of every Java application. Each keyword has special significance.

- **`public`**: Means the JVM can access the method from anywhere. If `main` is not public, the JVM cannot invoke it.
- **`static`**: Allows method execution WITHOUT object creation. The JVM calls `Main.main()` directly using the class name.
- **`void`**: Indicates the method returns nothing. The JVM does not expect a return value.
- **`main`**: Special predefined method name recognized by the JVM. Execution begins from `main()`.
- **`String args[]`**: Stores command-line arguments. For example, if you run `java Main hello`, then `args[0] = "hello"`.

🌟 Complete Example

```
class Main {
    public static void main(String args[]) {
        System.out.println("Program Started");
```

```
    }
}
```

🖥️ Output

```
Program Started
```

🎯 Conclusion

The `main` method acts as the execution entry point of Java applications, and every keyword in its declaration has a specific runtime purpose 🚀.